

ECE 4122 / 6122 — Advanced Programming Techniques

Homework 3: 3D World Scene Rendering with OpenGL and Assimp

OBJ File Loading · Lighting · Interactive Camera

Georgia Institute of Technology
School of Electrical and Computer Engineering

Due: See Canvas

Total Points: 100 | Language: C++ with OpenGL 3.3 Core Profile

Learning Objectives

- Load and render multiple OBJ 3D mesh files using the Assimp library
- Implement a scene graph positioning objects at specified world coordinates
- Apply Phong lighting (ambient + diffuse + specular) with at least two light sources
- Implement a first-person camera controlled via WASD keyboard and mouse
- Apply textures loaded from image files to mesh surfaces
- Compose a coherent outdoor scene with meaningful object placement

1. Assignment Overview

In this assignment you will build a real-time 3D scene viewer using OpenGL 3.3 Core Profile and the Open Asset Import Library (Assimp). Your program will load at least five distinct 3D mesh files in OBJ format, place them at different positions, orientations, and scales in a shared world coordinate system, and render the scene with lighting and texturing.

The theme of your scene is a rural farmstead environment, though you have creative latitude in the exact objects and layout you choose. The intent is to gain hands-on experience with the full real-time rendering pipeline: geometry loading → VAO/VBO setup → GLSL shader programming → lighting → texturing → camera control.

Platform Requirements

You **MUST** use OpenGL 3.3 Core Profile (GLSL version 330 core).

Do **NOT** use any legacy fixed-function calls: glBegin/glEnd, glLight*, glMatrixMode, etc.

All geometry must go through VAOs and VBOs. All shading must be done in GLSL shaders.

GLAD is the required OpenGL function loader (pre-generated files provided in src/glad/).

GLFW 3.4 is required for windowing and input. Assimp 5.x is required for model loading.

2. Detailed Requirements

2.1 Scene Composition (30 points)

Your scene must contain at least five (5) distinct OBJ mesh files placed at unique world positions. The recommended theme is a small rural farmstead, but you may choose an alternative theme.

Required placement properties for the complete scene:

- Position — each object must have a distinct translation (x, y, z) in world space
- Rotation — at least three objects must have a non-zero rotation applied
- Scale — at least two objects must be non-uniformly scaled (different x/y/z factors)
- Layout — objects must be logically arranged and must not all cluster at the origin

A flat ground plane (subdivided quad mesh) must be present and textured with a repeating grass or dirt texture.

2.2 Assimp Mesh Loading (25 points)

Implement Mesh and Model classes that use Assimp to load OBJ files. Your loader must handle:

- Vertices (positions), normals, and texture coordinates extracted from aiMesh
- Multiple meshes per model file — aiScene may contain multiple aiMesh nodes
- Diffuse texture maps from material data — loaded with stb_image into a GL texture
- Correct recursive traversal of the Assimp aiNode scene graph

Your Mesh class must populate a VAO with: position (location 0), normal (location 1), and texcoord (location 2). Your Model class must call the Assimp Importer and iterate all scene nodes recursively.

2.3 Lighting (20 points)

Implement shading entirely in GLSL. Your scene must include:

- One directional light (the sun) with configurable direction, ambient, diffuse, and specular components
- Per-vertex normals transformed to world space using the normal matrix
- Specular highlights that respond visibly as the camera moves

2.4 Interactive Camera (15 points)

Implement a free-fly (FPS-style) camera controlled by keyboard and mouse:

- W / A / S / D — move forward, left, backward, right along the camera's XZ plane
- Mouse motion — rotate camera yaw (left/right) and pitch (up/down)
- Scroll wheel — increase or decrease movement speed
- ESC — exit the application
- View matrix must be computed from position, front, and up vectors via glm::lookAt

2.5 Texturing (10 points)

At least three of your five objects must display a diffuse texture map loaded via `stb_image`. The ground plane must use a tiled texture. Texture coordinates must be passed to the fragment shader and sampled from a `sampler2D` uniform.

2.6 Code Quality & Submission (bonus up to 10 points)

- Clean, well-commented C++ code organised into logical header / source files
- A working `CMakeLists.txt` that builds successfully with `cmake + make` (or `ninja`)
- A `README.md` describing build steps, controls, and which assets you used
- Extra credit: skybox (cubemap), fog, or shadow mapping

3. Starter Code & Architecture

3.1 Recommended File Structure

```
hwk3/
├── CMakeLists.txt
├── README.md
├── src/
│   ├── main.cpp           // GLFW window, render loop, scene graph
│   ├── shader.h / .cpp    // compile & link GLSL programs
│   ├── mesh.h / .cpp      // VAO/VBO/EBO for one aiMesh
│   ├── model.h / .cpp     // Assimp OBJ loader; owns list of Meshes
│   ├── camera.h / .cpp    // free-fly FPS camera (view matrix)
│   └── texture.h / .cpp    // stb_image loader with gray fallback
├── shaders/
│   ├── object.vert
│   └── object.frag
├── assets/                // OBJ + MTL + texture files
│   ├── farmhouse/         farmhouse.obj farmhouse.mtl
│   │   └── Textures/      // Texture files
│   ├── tree/              tree.obj tree.mtl
│   └── ...
```

3.4 Vertex Shader (shaders/object.vert)

```
#version 330 core
layout(location = 0) in vec3 aPos;
layout(location = 1) in vec3 aNormal;
layout(location = 2) in vec2 aTexCoords;

out vec3 FragPos;    // world-space fragment position
out vec3 Normal;     // world-space normal
out vec2 TexCoords;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;
uniform mat3 normalMatrix; // = inverse(transpose(mat3(model)))

void main() {
    vec4 worldPos = model * vec4(aPos, 1.0);
    FragPos       = vec3(worldPos);
    Normal        = normalMatrix * aNormal;
    TexCoords     = aTexCoords;
    gl_Position   = projection * view * worldPos;
}
```

3.5 Fragment Shader (shaders/object.frag)

```
#version 330 core
out vec4 FragColor;
in  vec3 FragPos; in  vec3 Normal; in  vec2 TexCoords;

struct DirLight { vec3 direction, ambient, diffuse, specular; };
struct PointLight {
    vec3 position, ambient, diffuse, specular;
    float constant, linear, quadratic;
};

uniform DirLight sun;
uniform PointLight lantern;
uniform vec3 viewPos;
uniform sampler2D diffuseMap;
uniform float shininess;

vec3 CalcDirLight(DirLight l, vec3 n, vec3 vd, vec3 tc) {
    vec3 ld = normalize(l.direction);
    float d = max(dot(n, ld), 0.0);
    float s = pow(max(dot(vd, reflect(-ld, n)), 0.0), shininess);
    return (l.ambient + d*l.diffuse + s*l.specular) * tc;
}

vec3 CalcPointLight(PointLight l, vec3 n, vec3 fp, vec3 vd, vec3 tc) {
    vec3 ld = normalize(l.position - fp);
    float d = max(dot(n, ld), 0.0);
    float s = pow(max(dot(vd, reflect(-ld, n)), 0.0), shininess);
    float dst = length(l.position - fp);
    float att = 1.0 / (l.constant + l.linear*dst + l.quadratic*dst*dst);
    return att * (l.ambient + d*l.diffuse + s*l.specular) * tc;
}

void main() {
    vec3 n = normalize(Normal);
    vec3 vd = normalize(viewPos - FragPos);
    vec3 tc = vec3(texture(diffuseMap, TexCoords));
    FragColor = vec4(
        CalcDirLight(sun, n, vd, tc) +
        CalcPointLight(lantern, n, FragPos, vd, tc), 1.0);
}
```

3.6 Model Loader Skeleton

```
// model.h
class Model {
public:
    explicit Model(const std::string& path);
    void Draw(Shader& shader);
private:
    std::vector<Mesh> meshes;
    std::string directory;
    void ProcessNode(aiNode*, const aiScene*);
    Mesh ProcessMesh(aiMesh*, const aiScene*);
};

// model.cpp - ProcessMesh core snippet
Mesh Model::ProcessMesh(aiMesh* mesh, const aiScene*) {
    for (unsigned i = 0; i < mesh->mNumVertices; ++i) {
        Vertex v;
        v.Position = { mesh->mVertices[i].x,
                       mesh->mVertices[i].y,
                       mesh->mVertices[i].z };
        v.Normal = { mesh->mNormals[i].x,
                    mesh->mNormals[i].y,
                    mesh->mNormals[i].z };
        if (mesh->mTextureCoords[0])
            v.TexCoords = { mesh->mTextureCoords[0][i].x,
                           mesh->mTextureCoords[0][i].y };
        vertices.push_back(v);
    }
    // extract indices from mesh->mFaces ...
    // load diffuse texture via aiMaterial ...
}
```

3.7 Assimp Import Flags

Use these post-processing flags when calling `importer.ReadFile()`:

```
unsigned int flags =
    aiProcess_Triangulate           // convert all polygons to triangles
| aiProcess_FlipUVs               // flip V coordinate for OpenGL
| aiProcess_GenSmoothNormals      // generate normals if absent
| aiProcess_CalcTangentSpace;     // pre-computes tangents (for future normal mapping)

const aiScene* scene = importer.ReadFile(path, flags);
if (!scene || (scene->mFlags & AI_SCENE_FLAGS_INCOMPLETE)
    || !scene->mRootNode)
    throw std::runtime_error(importer.GetErrorString());
```

3.8 Scene Placement in main.cpp

```

struct SceneObject {
    Model*    model;
    glm::vec3 position;
    float     rotationDeg;
    glm::vec3 rotationAxis;
    glm::vec3 scale;
    float     shininess;
};

std::vector<SceneObject> scene = {
    {&farmhouse, { 0.f,0.f,-2.f}, 0.f,{0,1,0},{1.5f,1.5f,1.5f},16.f},
    {&treel, { 5.f,0.f, 2.f}, 15.f,{0,1,0},{1.0f,1.0f,1.0f}, 8.f},
    {&barn, {10.f,0.f, 1.f}, 45.f,{0,1,0},{2.0f,2.0f,2.0f},32.f},
    {&lamp, { 3.f,0.f,-0.5f}, 0.f,{0,1,0},{0.6f,0.6f,0.6f},64.f},
    {&fence, {-2.f,0.f, 3.f}, 90.f,{0,1,0},{2.5f,1.0f,1.0f},16.f},
};

// In the render loop:
for (auto& obj : scene) {
    glm::mat4 M = glm::translate(glm::mat4(1.f), obj.position);
    M = glm::rotate(M, glm::radians(obj.rotationDeg), obj.rotationAxis);
    M = glm::scale (M, obj.scale);
    glm::mat3 NM = glm::transpose(glm::inverse(glm::mat3(M)));
    shader.SetMat4 ("model", M);
    shader.SetMat3 ("normalMatrix", NM);
    shader.SetFloat("shininess", obj.shininess);
    obj.model->Draw(shader);
}

```

4. Recommended 3D Assets

You must find and download at least five OBJ files. The table below lists free assets known to work well with Assimp and having permissive licences. All fit the rural/farmstead theme; alternatives may be substituted with prior approval.

Object	Source	URL / Notes
Wooden Cottage / Cabin	Sketchfab (free)	sketchfab.com → search 'low poly cabin' → Download OBJ
Pine / Fir Tree	Free3D.com	free3d.com → search 'pine tree' → OBJ format
Street Lamp / Lantern	TurboSquid (free)	turbosquid.com → search 'street lamp' → filter Free
Wooden Fence Section	CGTrader (free)	cgtrader.com → search 'wooden fence' → filter Free
Rustic Barn	Sketchfab (free)	sketchfab.com → search 'barn' → filter Downloadable
Pickup Truck	Free3D.com	free3d.com → search 'pickup truck' → OBJ
Hay Bale	CGTrader (free)	cgtrader.com → search 'hay bale'
Stone Well	Sketchfab (free)	sketchfab.com → search 'stone well OBJ'
Mailbox	Sketchfab (free)	sketchfab.com → search 'mailbox OBJ'
Wooden Bench	Free3D.com	free3d.com → search 'park bench OBJ'

Windmill	Sketchfab (free)	sketchfab.com → search 'windmill'
Rock / Boulder Set	Poly Haven (CC0)	polyhaven.com/models — CC0, no attribution required

Important notes when downloading:

- Always choose OBJ format (not FBX or GLB) unless you have verified Assimp loads it
- Confirm a .mtl file is included alongside the .obj — it carries material and texture references
- Textures should be PNG or JPG; TGA and DDS require extra stb_image compile-time flags
- Poly Haven (polyhaven.com) provides CC0 assets — no attribution required
- Sketchfab free downloads require a free account; filter by licence 'CC Attribution' or 'CC0'
- Inspect each file in MeshLab or Windows 3D Viewer before integrating it

Pro Tip: Normalising Model Scale

Downloaded models often arrive in wildly different scales — a tree may be 0.01 units tall while a cabin is 500 units. Use a viewer to inspect and rescale, or pass the `aiProcess_GlobalScale` post-processing flag to Assimp at import time. Alternatively, add a 'scale' field to your SceneObject struct and tune it visually.

6. Grading Rubric

Requirement	Points	Score
Scene Composition — 5+ distinct OBJ objects placed logically	30	
Objects have varied position, rotation, and scale	(15)	
Ground plane present and textured	(10)	
Scene is thematically coherent and visually appealing	(5)	
Assimp Mesh Loading — VAO/VBO/EBO, normals, texcoords correct	25	
Multi-mesh models and material textures handled	(10)	
Recursive aiNode scene-graph traversal	(15)	
Phong Lighting — directional sun + point lantern w/ attenuation	20	
Normal matrix computed and applied correctly	(10)	
Specular highlights respond to camera movement	(10)	
Interactive Camera — WASD + mouse, correct view matrix	15	
Texturing — 3+ textured objects, ground plane tiled	10	
Code quality, CMakeLists.txt builds cleanly, README present	Bonus +10	
TOTAL	100 + 10 bonus	

Note: Code that does not compile on the grader's machine using the provided CMakeLists.txt will receive a maximum of 50% credit. Always verify a clean build from a fresh build directory before submitting.

7. Submission Instructions

1. Create a ZIP archive named `Lastname_Firstname_HW3.zip`
2. The ZIP must contain: all source files, `CMakeLists.txt`, `shaders/`, `assets/`, and `README.md`
3. Do NOT include compiled binaries, the `build/` directory
4. Submit via the course Canvas page before the deadline
5. Include a 30–60 second screen-captured walkthrough video (.mp4) — upload to Canvas or include a YouTube/Drive link in your README

Academic Integrity Reminder

You are encouraged to reference online tutorials — LearnOpenGL.com is an excellent resource.

All code you submit must be written by you. Do not copy-paste complete class implementations from the web. You must be able to explain every line of your submission. The shader skeletons and `CMakeLists.txt` in Section 3 are provided as starting points and may be used directly.

Helpful References

- LearnOpenGL.com — Model Loading chapter (complete Assimp / Mesh / Model walkthrough)
- Assimp documentation — assimp.sourceforge.net/lib_html/
- stb_image — github.com/nothings/stb (single-header image loader)
- GLM documentation — glm.g-truc.net (vector and matrix math library)
- GLFW documentation — glfw.org/documentation.html
- MeshLab (free mesh viewer / converter) — meshlab.net
- Poly Haven (CC0 assets) — polyhaven.com/models
- Sketchfab (free downloads) — sketchfab.com